# Simple Conflict Resolution: "Wait Your Turn"

Exactly as implemented in `simple_resolver.py` (Dubins_TSP_New_Implementation_claude). No game theory: no players, costs or equilibrium. Numbers are the defaults; equation numbers refer to Dubins_TSP.pdf.

## 1. Conflict Detection

Positions $p_k[n]$ are sampled every $\Delta t = 0.5$ s; between samples the sinks move in straight lines. Each step from frame $n$ to $n + 1$ is looked at five times:

$$p_k(n,t) = p_k[n] + t\,(p_k[n+1] - p_k[n]), \qquad t \in \{0,\ 0.25,\ 0.5,\ 0.75,\ 1\}$$

Step $n$ is a conflict between sinks $i$ and $j$ ($u_{ij}[n] = 1$) if at any of these looks

$$\| p_i(n,t) - p_j(n,t) \| \ < \ d_s$$

with $d_s = 3$ m, unless both sinks are within 5 m of the base at that look. It is checked only while both sinks are on their mission (eq. 20).

## 2. Order of Turns

Sinks take turns by decreasing mission time (ties: lower index first):

$$T_{(1)} \ \geq \ T_{(2)} \ \geq \ \cdots \ \geq \ T_{(K)}$$

The longest tour has the least spare time, so it goes first and never waits. Each sink $k$ looks only at $D_k$, the sinks that have already had their turn; their trajectories are final. So every pair is handled once, by the sink that comes later in the order.

## 3. First Meeting

Starting from $n_0 = 1$, sink $k$ finds its first conflict with a sink in $D_k$. If there is none, its turn ends.

$$f = \min \ \{\ n \geq n_0 \ :\ u_{kj}[n] = 1 \text{ for some } j \in D_k \ \}$$

## 4. The Stop

The sink stops at frame $s = \max(1, f - 5)$, which is 2.5 s before the meeting, for $w$ frames. Its new trajectory is

$$P'[n] = P[n] \quad (n \leq s), \qquad P[s] \quad (s < n \leq s + w), \qquad P[n - w] \quad (n > s + w)$$

This adds $w\Delta t$ to the mission time $T_k$ and nothing to the path length. The stop is limited by 20 s and by the time left in the budget $B$:

$$w_{\max} = \min\,(\,40,\ \lfloor\,(B - T_k)\,/\,\Delta t\,\rfloor\,)$$

The sink takes the shortest stop after which the way is clear, from the stop until 5 s (10 frames) after it reaches the meeting point:

$$w^* = \min \ \{\ w \leq w_{\max} \ :\ u'_{kj}[n] = 0 \ \text{ for all } j \in D_k, \quad s \leq n < f + w + 10 \ \}$$

Here $u'$ is the conflict test with the new trajectory $P'$. If $w^*$ exists, the stop is applied and the search continues from $n_0 = f + w^*$. If not, this conflict is left in place and the search continues from $n_0 = f + 1$.

## 5. Why It Always Stops

Each sink has exactly one turn. Within a turn $n_0$ grows after every meeting, while the trajectory can grow by at most $\lfloor (B - T_k)/\Delta t \rfloor$ frames in total, so the search reaches its end after finitely many steps.

## 6. What Holds for Every Input

- **Path unchanged:** a stop only repeats positions, so the path length, obstacle clearance (eq. 19) and turning radius stay as planned.
- **Budget (eq. 18):** a stop never pushes $T_k$ over $B$.
- **Priority:** the longest tour never changes, and a sink never changes after its turn.

**Important limitation:** the result is not guaranteed to be conflict-free or to have the least waiting. Waiting cannot solve a head-on meeting on one line (the other sink drives through the stopped one), and a sink with no time left in its budget cannot wait. The order is fixed and never optimised. Away from the base, five looks per step can miss only a pass whose closest distance is between 2.93 m and 3 m. Conflicts that remain are reported by the separate closest-distance check that `planner.py` runs, not hidden.